

Accelerate TarFlow Sampling with GS-Jacobi Iteration

Ben Liu ^{*,2} and Zhen Qin [†]

¹TapTap, Shanghai, China

²Zhejiang University, Hangzhou, China

Abstract

Image generation models have achieved widespread applications. As an instance, the TarFlow model combines the transformer architecture with Normalizing Flow models, achieving state-of-the-art results on multiple benchmarks. However, due to the causal form of attention requiring sequential computation, TarFlow’s sampling process is extremely slow. In this paper, we demonstrate that through a series of optimization strategies, TarFlow sampling can be greatly accelerated by using the Gauss-Seidel-Jacobi (abbreviated as GS-Jacobi) iteration method. Specifically, we find that blocks in the TarFlow model have varying importance: a small number of blocks play a major role in image generation tasks, while other blocks contribute relatively little; some blocks are sensitive to initial values and prone to numerical overflow, while others are relatively robust. Based on these two characteristics, we propose the Convergence Ranking Metric (CRM) and the Initial Guessing Metric (IGM): CRM is used to identify whether a TarFlow block is "simple" (converges in few iterations) or "tough" (requires more iterations); IGM is used to evaluate whether the initial value of the iteration is good. Experiments on four TarFlow models demonstrate that GS-Jacobi sampling can significantly enhance sampling efficiency while maintaining the quality of generated images (measured by FID), achieving speed-ups of 4.53× in Img128cond, 5.32× in AFHQ, 2.96× in Img64uncond, and 2.51× in Img64cond without degrading FID scores or sample quality. Code and checkpoints are accessible on https://github.com/encoreus/GS-Jacobi_for_TarFlow.

1 Introduction

Image generation models have been widely applied in various scenarios. As an instance, normalizing-flow-based models, from the original NICE [Dinh et al., 2014] model, to improved RealNVP [Dinh et al., 2016] and Glow [Kingma and Dhariwal, 2018] models, offer unique advantages through their invertible architecture that applies sequence of lossless transformations to noise vectors, but show limited performance in the generation of high solution and complex images.

Recently, Transformer Autoregressive Flow model (TarFlow, [Zhai et al., 2024]) introduces stacks of autoregressive Transformer blocks (similar to MAF [Papamakarios et al., 2017]) into the building of affine coupling layers to do Non-Volume Preserving, combined with guidance [Ho and Salimans, 2022] and denoising [Bigdeli et al., 2023], finally achieves state-of-the-art results across multiple benchmarks. However, TarFlow’s sampling efficiency suffers from a critical bottleneck: the causal attention structure within each autoregressive block forms a nonlinear RNN. This forces strictly sequential computation during sampling, where each step must wait for the full update of preceding key-value caches, resulting in significantly reduced computational efficiency for large-scale image generation tasks.

*liuben0330@gmail.com

†zhenqin950102@gmail.com

In this paper, we try to solve this. We first transform the nonlinear RNN in the TarFlow sampling phase into a diagonalized nonlinear system, then we can employ iteration-based solvers such as Gauss-Seidel-Jacobi (GS-Jacobi) iteration [Ortega and Rheinboldt, 2000] [Song et al., 2021] [Santilli et al., 2023]. However, naively applying GS-Jacobi iteration leads to generation failure (see 1st and 2nd row of Figure 4). Through detailed analysis, we discover that blocks in the TarFlow model have varying importance: a small number of blocks play a major role in image generation tasks, while other blocks contribute relatively little; some blocks are sensitive to initial values and prone to numerical overflow, while others are relatively robust. Based on these two characteristics, we propose the Convergence Ranking Metric (CRM) and the Initial Guessing Metric (IGM): CRM is used to identify whether a TarFlow block is "simple" (converges in few iterations) or "tough" (requires more iterations); IGM is used to evaluate whether the initial value of the iteration is appropriate. Leveraging these two metrics, we present a sampling algorithm that substantially reduces calculation amount required.

As a summary, we list our contributions as follows:

- Transform the sampling process of the TarFlow model into a diagonalized nonlinear system, and apply a Gauss-Seidel-Jacobi hybrid iteration scheme to it, providing corresponding error propagation analysis and convergence guarantees;
- To identify the non-uniform transformation patterns in TarFlow blocks and control the number of iterations, we propose the Convergence Ranking Metric (CRM) to evaluate and measure them;
- To control the stability of iteration initial values to avoid numerical overflow, we propose the Initial Guessing Metric (IGM);
- Comprehensive experiments demonstrate $4.53\times$ speedup in `Img128cond`, $5.32\times$ in `AFHQ`, $2.96\times$ in `Img64uncond`, $2.51\times$ in `Img64cond` sampling without measurable degradation in FID scores or sample quality.

2 Related Work

Normalizing Flow Based Models In the field of image generation, numerous methods have been proposed. From PixelRNN ([Van Den Oord et al., 2016]), GANs [Goodfellow et al., 2020], to DDPM [Ho et al., 2020], Stable Diffusion ([Podell et al., 2023]). Diffusion models seem to dominate this field, but normalizing flows still offer unique advantages, including exact invertibility [Whang et al., 2021] enabling precise density estimation [Trippe and Turner, 2018], single-step sampling for efficient generation [Grcić et al., 2021], and structured latent spaces that support interpretable manipulation [Bose et al., 2020].

Normalizing Flows learn an invertible model f that transforms noise z into data x , such that $x = f(z)$. The key to build invertible models is accessible inverse function with Jacobi determinant easy to calculate, series of flow models accomplishes this through coupling layers. NICE [Dinh et al., 2014] introduced the additive coupling layers. To enhance the non-linear capability, RealNVP [Dinh et al., 2016] integrated scaling and shifting to the non-volume preserving transform as the affined coupling layer. Glow [Kingma and Dhariwal, 2018] improved the images generation by introducing invertible 1×1 convolution, and Flow++ [Ho et al., 2019] included attention mechanic. The most significant advantage of these models is that the inverse function is explicit and Jacobi matrix is lower triangle. This can avoid the complex calculation in the general invertible ResNet framework proposed in [Behrmann et al., 2019]. However, overly simple structure makes these flow models less nonlinear.

To improve this, normalizing flows are combined with autoregressive models. IAF [Kingma et al., 2016] pioneered dimension-wise affine transformations conditioned on preceding dimensions to improve variational inference. MAF [Papamakarios et al., 2017] utilized the MADE [Germain et al., 2015] to create invertible autoregressive mappings. NAF [Huang et al., 2018], which replaced MAF's affine transformations with per-dimension monotonic neural networks to enhance expressivity. T-NAF [Patacchiola et al., 2024] augmented NAF by integrating a single autoregressive Transformer, whereas Block Neural Autoregressive Flow [De Cao et al., 2020] adopted an end-to-end autoregressive monotonic network design. TarFlow [Zhai et al., 2024] proposed a Transformer-based architecture together with a set of techniques to train high performance normalizing flow models and show SOTA in many fields, thus becomes the sampling object of this article.

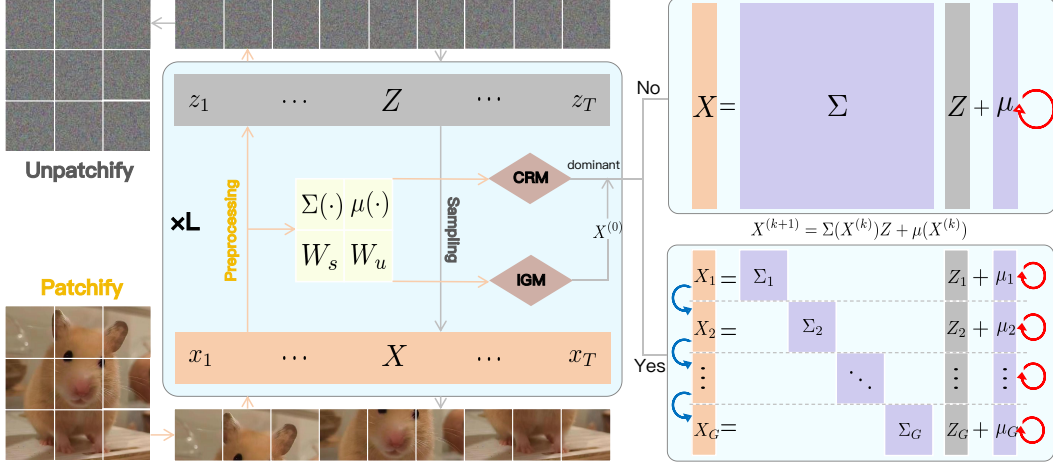


Figure 1: Simple intuition diagram of GS-Jacobi sampling. First pass forward a small batch of images to compute Initial Guessing Metric (IGM) and Convergence Ranking Metric (CRM) for each block. When sampling, the initial iteration value $X^{(0)}$ is determined by IGM; for blocks whose CRM is non-dominant, parallel Jacobi iterate X ; for CRM-dominant blocks, segment X into small modules X_g , parallel Jacobi iterating within modules, then serially deliver to next module.

Parallel solving of linear/nonlinear systems Linear/nonlinear systems refer to $f_i(\mathbf{x}) = 0, \mathbf{x} \in \mathbb{R}^n$, where $f_i, i = 1, \dots, n$ is a linear/nonlinear function. The parallel solution of these systems is an important problem in scientific computing. [Saad, 2003] established methods such as Jacobi, Gauss-Seidel, successive over-relaxation (SOR), and Krylov subspace techniques for linear systems. Block-Jacobi iterations [Anzt et al., 2015, 2016, Chow et al., 2018] use GPU parallelization to solve linear/nonlinear equations. As a special case, when f_i takes the form $f_i(\mathbf{x}) = \mathbf{x}_i - g_i(\mathbf{x}_{<i})$, it is called an autoregressive system. Lots of approaches have been proposed to accelerate autoregressive computation. [Oord et al., 2018] introduced probability density distillation for transferring knowledge from slow autoregressive models to faster computation. MintNet [Song et al., 2019] developed a specialized Newton-Raphson-based fixed-point iteration method to speed up autoregressive inversion. Similar theoretical concepts were earlier explored by [Naumov, 2017] without empirical validation.

3 Methods

3.1 Jacobi Mode Fixed Point Iteration Sampling

Let z denotes the noise direction and x denotes the image direction, both with size (B, T, C) , where B, T, C represent batch size, patchified sequence length, and feature dimension, respectively. For TarFlow model, an autoregressive block can be written as:

$$\begin{aligned} \text{Forward: } z_t &= \exp(-s(x_{<t}))(x_t - u(x_{<t})), \\ \text{Inverse: } x_t &= \exp(s(x_{<t}))z_t + u(x_{<t}). \end{aligned} \quad (1)$$

for $t = 1, \dots, T$ and $x_1 = z_1$. $x_{<t} := \{x_i\}_{i=1}^{t-1}$ denotes the history before time t , $s(x_{<t}), u(x_{<t})$ generated from causal attention block. In forward direction, all x_t are given, so all $s(x_{<t}), u(x_{<t})$ can be calculate in parallel. But in inverse direction, x_t can only be computed serially after $x_{<t}$ has been solved. In Table 3a, it takes about 213 seconds for such serial sampling to generate 100 128×128 images with a single A800 GPU.

Denote $\exp(-s(x_{<t})) = \sigma_t^{-1}, u(x_{<t}) = u_t$, the former process can be written in a matrix form:

$$\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_T \end{bmatrix} = \begin{bmatrix} \sigma_1^{-1} & & & \\ & \sigma_2^{-1} & & \\ & & \ddots & \\ & & & \sigma_T^{-1} \end{bmatrix} \left(\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_T \end{bmatrix} - \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_T \end{bmatrix} \right). \quad (2)$$

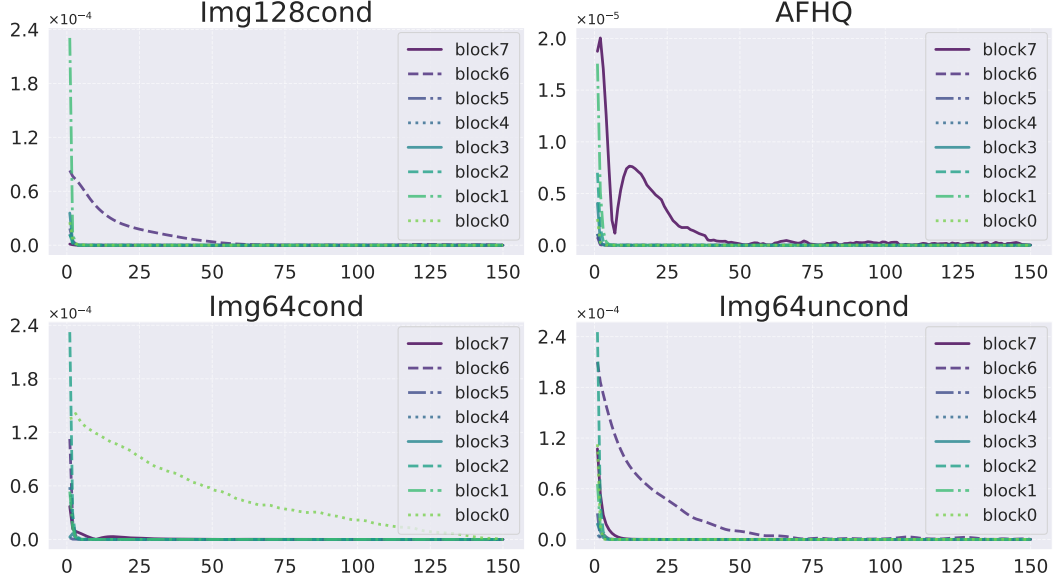


Figure 2: The distance between $X^{(k)}$ (up to 150 times) and target X^* of all 8 blocks in four models. Most blocks converge within iteration times $\ll T$, with each model exhibiting only one or two slowly descending curves.

with $\sigma_1 = 1, u_1 = 0$, then the transform from X to Z can be seen as an non-linear system:

$$\begin{aligned} \text{Forward: } Z &= \Sigma^{-1}(X)(X - \mu(X)), \\ \text{Inverse: } X &= \Sigma(X)Z + \mu(X). \end{aligned} \quad (3)$$

For the inverse process, we can view the target X^* as the fixed point of the nonlinear system $g(X) = \Sigma^{-1}(X)Z + \mu(X)$, and then solve it using the non-linear Jacobi iteration [Kelley, 1995]:

$$\begin{aligned} X^{(k+1)} &= \Sigma(X^{(k)})Z + \mu(X^{(k)}), \\ x_t^{(k+1)} &= \sigma_t^{(k)} z_t + u_t^{(k)}, \text{ parallel for } t = 1, \dots, T. \end{aligned} \quad (4)$$

with an initialized $X^{(0)}$. We propose Proposition (1) to explain the convergence and error propagation of Jacobi mode iteration under this nonlinear system. See Appendix A for detailed discussion.

Proposition 1 (Converge and Error Propagation). *For fixed point iteration (4), let $\varepsilon^{(k)} = X^{(k)} - X^*$ be the error after k iteration, e_t be its t -th component, $f_t^{(k)} = x_t^{(k)} - \sigma_t^{(k)} z_t - u_t^{(k)}$, then:*

- Equation (4) converges strictly after $T - 1$ times,
- $e_t^{(k)} \approx -\sum_{i=k+1}^{t-1} \gamma_{ti}^{(k)} e_i^{(k)}$, with $\gamma_{ti}^{(k)} = \frac{\partial f_t}{\partial x_i} \big|_{X^{(k)}}$, $t \geq k + 2$.

This iteration method involves two components: the initial value $X^{(0)}$ and the maximum number of iterations. As shown in Figure 4, different initialization strategies lead to different convergence effects, with poor strategies causing model collapse. Also as shown in Figure 2, different blocks converge at varying speeds, some blocks converge quickly while others slowly, suggesting that we should employ different iteration strategies for different blocks. We propose Initial Guessing Metric and Convergence Ranking Metric to address these two issues, respectively.

3.2 Initial Guessing Metric

A common choice for initialization is to take $X^{(0)} = Z = [z_1, z_2, \dots, z_t]'$, i.e, the output of the former block, with intuition TarFlow transform images "gradually" (Zhai et al. [2024]), which means that the difference between adjacent TarFlow blocks maintain stable. As shown in Figure 3, the change from noise to image in most steps is gradual, and Z locates in the neighbor of X^* , which



Figure 3: The trace of the sampling in four models. From top to bottom: Img128cond, Img64cond, Img64uncond, AFHQ. From left to right: noise, Block 7-0, denoised image.

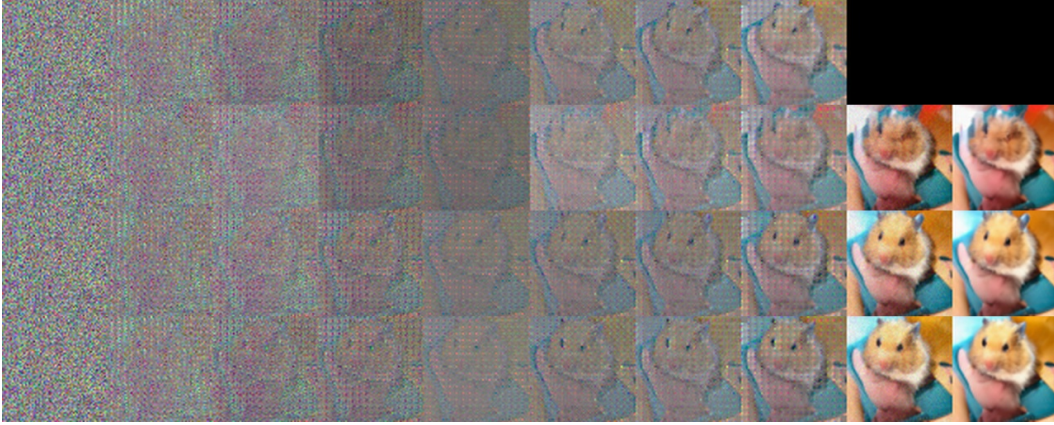


Figure 4: The influence of different initial value and iteration times of an Img64cond sample. From top to bottom: Set all $X^{(0)} = Z$, Jacobi 30 times; Adaptive initialized according to IGM, Jacobi 20 times; Adaptive by IGM, Jacobi 30 times; GS-Jacobi [0/7-16/8-10/13-6]

can be a good initial guessing. However, in practice, we find that take all $X^{(0)} = Z$ cause numeric collapse in Img64 models in Block0, as shown in the 1st row of Figure 4.

An alternative workable guessing is $X^{(0)} = Z_0 = [z_1, 0, \dots, 0]'$, since pixel value  from -1 to 1 and centers in 0. A natural strategy is comparing Z and Z_0 and choose the better one. Since the worst inflation occurs at first few iteration, we define following "Initial Guessing Metric":

$$\text{IGM}(X^{(0)}) = \|\Sigma(X^{(0)})Z + \mu(X^{(0)}) - X^*\|_2. \quad (5)$$

to measure a rough distance with $X^{(0)}$ chosen from $\{Z, Z_0\}$. In Appendix D, different norms showed similar results, spectral norm is a little better and we use it in this paper. 

We can treat IGM as a model property which is determined once training completed. So we don't need to repeatedly calculate (5) each time sampling and instead use the following steps:

- Select a batch of images from the training set, patching to size (B, T, C) , which is X^* ;
- Forward passing X^* through TarFlow blocks to get $Z = \Sigma^{-1}(X^*)(X^* - \mu(X^*))$;
- Calculate the residual $\Sigma(X^{(0)})Z + \mu(X^{(0)}) - X^*$ with both $\{Z, Z_0\}$;
- Calculate the mean of residual in the dim B , calculate the norm of the (T, C) matrix.

3.3 Convergence Ranking Metric

When sampling with (4), although all blocks converge strictly, some get nice solution with very small k , while others need k near $T - 1$. As shown in Figure 2, Block6 of Img128cond, Block7 of AFHQ, Block0 of Img64cond, Block6 of Img64uncond behave worse compared to other blocks. To measure this difference, we propose the following Convergence Ranking Metric:

$$\text{CRM} = \|\Sigma^{-1}(X)X\|_2 \|W_s\|_2 + \|W_u\|_2 \quad (6)$$

with W_s, W_u the weight matrix of the project out layer of $s(x_{<t}), u(x_{<t})$. W_s measures the change of variance; W_u measures the mean, and $\Sigma^{-1}(X)X$ measures the non-volume-preserving. See Appendix B for detailed derivation. CRMs can be calculated with the following steps:

- Extract the project out parameters for each TarFlow block, calculate $\|W_s\|_2, \|W_u\|_2$;
- Select a small batch of images from the training set, go through the forward process, get $\Sigma^{-1}(X)X$ with size (B, T, C) , take means over B dimension to get (T, C) size matrixs;
- Calculate $\|\Sigma^{-1}(X)X\|_2$ then CRM for each block.

This metric doesn't strictly measure the convergence rate, only represents the relative convergence ranking among TarFlow blocks, therefore we call it a ranking metric. In Appendix D, different matrix norms behave similarly in relatively ranking, and we use spectral norm.

By CRM, we can know whether a block can converge rapidly or slowly. It roughly determine the iteration times of (4). Blocks with dominant CRM values in Table 2 converge slowly in Figure 2. In practice, although only very few blocks in TarFlow converge slowly, this severely affects the speed and effectiveness of the Jacobi iteration method: For "tough" blocks, fewer iterations result in poor generation quality (see 2nd row of Figure 4), while more iterations improve the model but simultaneously lose the speed advantage. As shown in Table 3a 3b, the Jacobi-30 strategy exhibits significantly inferior performance, while Jacobi-60 shows measurable improvement with much more time cost.

3.4 Modular Guass-Seidel-Jacobi Iteration

For a (B, T, C) tensor, (4) updates all T units in parallel, while "For" iteration updates 1 unit a time, serially run $T - 1$ times. Naturally, an in-between method is to update a set of units in parallel (with Jacobi) in one iteration, and serially go to another set, that's so-called Guass-Seidel-Jacobi iteration. Let $X := \{x_t\}_{t=1}^T, \{\mathcal{G}_g\}_{g=1}^G$ an non-decrease segmentation for time-step index $1 : T$, $X_g := \{x_t | t \in \mathcal{G}_g\}_{g=1}^G, X_{:g} := \bigcup_{i=1}^g X_i$, and similar definition for $\{Z, z_t\}, \{\Sigma, \sigma_t\}, \{\mu, u_t\}$. Then the concept of modular GS-Jacobi method can be shown in Figure 1, and detailed algorithm is shown in Appendix E.

All the analysis of Jacobi mode iteration is applicable to the modules of GS-Jacobi sampling. We point out that GS-Jacobi can effectively improve the solution for blocks with large CRM:

- The probability of numerical overflow due to initial guessing value is greatly reduced. The size of error matrix (7) is smaller thus the error cumsum (8) reduced;
- The convergence of each sub-Jacobi will be accelerated, since the modules closer to the back will have a more accurate initial value;
- An appropriate GS-Jacobi strategy (select $\mathcal{G}_g$ and maximum Jacobi iteration times) can achieve both accurate and fast solution.

We segment the tough blocks into 8 equal modules and apply GS-Jacobi iteration in Figure 5. In Figure 2, pure Jacobi iteration requires between 50 to 150 times to converge for tough blocks, whereas in Figure 5, the GS-Jacobi method reduces this number to approximately 30, and usually only module1 suffer a more difficult trace.

So, a proper strategy can take advantage of such modular iteration method. Ideally, IGM and CRM should be calculated for each GS-Jacobi modules to judge it is tough or not. Then for every modules, allocate more iteration to large CRM and vice versa. This can be seen as an adaptive strategy.

In practice, equal-size segmentation and same Jacobi times is usually enough. Then strategies can be denoted in the format [Stack-GS-J-Else]. Stack indicates the tough blocks should be segmented; GS

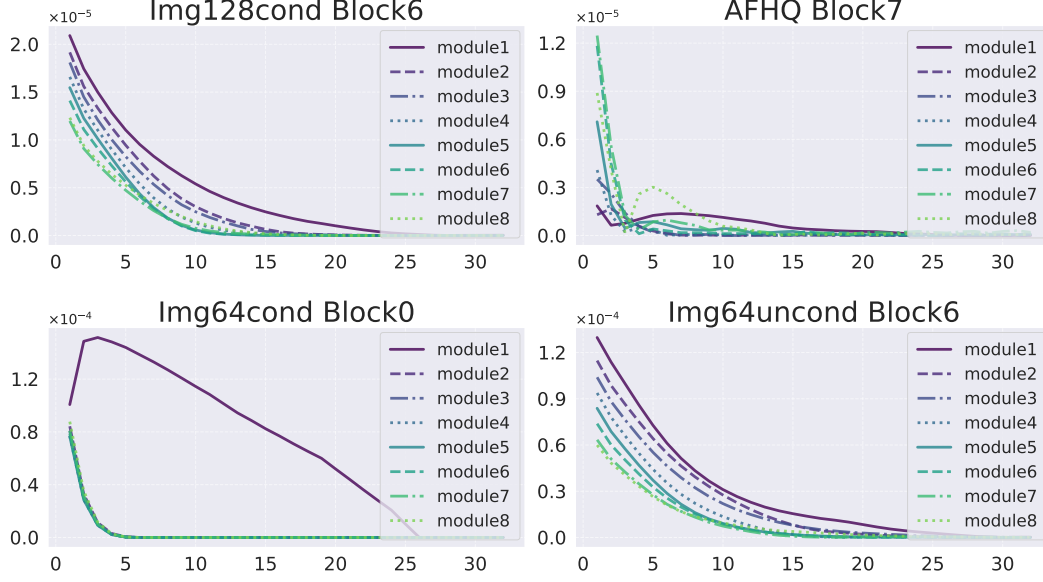


Figure 5: The distance between GS-Jacobi iteration and target X_g^* of four tough blocks. All modules tend to converge within 30 iterations, and the 1st module suffer a more difficult trace.

indicates the number of equal size segmentation with length $T//GS$; **J** indicates the maximum Jacobi times of each module; **Else** indicates the maximum Jacobi times for other blocks with small CRM.

To determine the stacked blocks, select blocks with large CRM one by one until there are no dominant blocks in the remaining set. By Table 2, we stack Block6 in Img128cond, Block7 in AFHQ, Block0&6 in Img64uncond, Block0&7 in Img64cond.

4 Experiment

We train four models given by [Zhai et al., 2024]: TARFLOW [4-1024-8-8- $\mathcal{N}(0, 0.15^2)$] for Conditional ImageNet 128×128 (Deng et al. [2009]); TARFLOW [8-768-8-8- $\mathcal{N}(0, 0.07^2)$] for AFHQ 256×256 (Choi et al. [2020]); TARFLOW [2-768-8-8- $\mathcal{N}(0, 0.05^2)$] for Unconditional ImageNet 64×64 (Van Den Oord et al. [2016]); TARFLOW [4-1024-8-8- $\mathcal{N}(0, 0.05^2)$] for Conditional ImageNet 64×64. The first three $T = 1024$, the last $T = 256$, and all four models have 8 TarFlow blocks. For convenience we will refer to them as Img128cond, AFHQ, Img64uncond and Img64cond.

4.1 Initial Guessing Metric

We first calculate IGMs for four models with 128 training images, as shown in Table 1. We find that there are not significant difference between two initializations in Img128cond and AFHQ, while the sampling of Img64cond and Img64uncond will collapse if initialize $X^{(0)} = Z$ for all blocks. This is evident in Table 1 since IGMs of Block0 in Img64cond and Img64uncond are pathological with $X^{(0)} = Z$, while set $X^{(0)} = Z_0$ can release this.

As shown in Figure 2, IGM is highly correlated with the potential maximum value occur during the iteration. We find that Img64cond and Img64uncond are more sensitive to the initial value. This may be because low-resolution images are more prone to mutations between pixels, which causes huge fluctuations in the attention layers parameters. In practice, the GS-Jacobi segmentation can greatly improve the problem of numerical overflow, so it is sufficient to simple initialize with Z, Z_0 by IGM.

4.2 Convergence Ranking Metric

We calculate CRMs for four models with 128 images in Table 2 the same time with IGMs. Detailed components are shown in Appendix C. Table 2 is consistent with Figure 2, following the simple rule: The larger the CRM, the more Jacobi times required for convergence, and vice versa.

选择[Z,Z0] 中 IRM更小的

Table 1: Initial Guessing Metric for initialization with Z, Z_0 for four models.

Blocks $X^{(0)}$	Img s128cond		Img64cond		Img64uncond		AFHQ	
	Z	Z_0	Z	Z_0	Z	Z_0	Z	Z_0
Block0	12.06	14.15	$5.4 \times 1e5$	9.89	1443.13	9.98	21.81	26.28
Block1	14.19	3.55	7.23	6.92	10.49	5.64	12.48	9.81
Block2	3.35	5.66	6.19	5.92	5.89	6.25	11.88	8.73
Block3	10.33	14.23	4.99	6.40	6.65	3.50	34.36	40.80
Block4	9.04	29.26	10.64	13.45	3.13	8.47	15.58	46.59
Block5	14.78	26.89	5.55	24.23	2.74	5.46	28.61	48.51
Block6	53.42	42.03	4.31	20.18	34.98	5.64	13.91	51.48
Block7	11.00	39.67	23.64	13.92	23.11	31.86	124.80	134.86

num_GS_list =
[0,1,1,1,1,1,1,1]

max_Jacobi_list =
[32,10,10,10,10,10,10,10]

Table 2: Convergence Ranking Metric of four TarFlow models, with dominant blocks bolded.

Models	Img128cond		AFHQ		Img64uncond		Img64cond	
	CRM	Percent	CRM	Percent	CRM	Percent	CRM	Percent
Block0	6.52	5.22	51.85	6.33	22.29	50.71	141.22	74.58
Block1	7.03	5.63	51.45	6.28	1.06	2.42	9.25	4.88
Block2	3.08	2.47	66.76	8.16	1.01	2.29	1.36	0.72
Block3	13.63	10.93	64.98	7.94	1.48	3.38	1.82	0.96
Block4	9.66	7.74	73.77	9.01	0.77	1.77	7.68	4.05
Block5	9.17	7.35	84.05	10.27	0.58	1.33	5.08	2.68
Block6	70.54	56.57	76.64	9.36	14.78	33.62	3.08	1.62
Block7	5.05	4.05	348.51	42.60	1.95	4.44	19.81	10.46

num_GS_list =
[1,8,1,1,1,1,1,1]

max_Jacobi_list =
[10,32,10,10,10,10,10,10]

num_GS_list =
[8,1,1,1,1,1,1,0]

max_Jacobi_list =
[13,6,6,6,6,6,6,0]

An important property is, only very few blocks in a TarFlow model have relative large CRM. This may be because TarFlow, or other normalizing-flow based generative models are over-determined, which means that the amount of parameters is redundant relative to the generative capacity, and many blocks don't modify the images drastically, only carefully crafted. As shown in Figure 3, visually, many middle blocks have no obvious changes, which provides the possibility of GS-Jacobi acceleration.

To identify such "tough" blocks, we just need to repeatedly select the block with the largest CRM until there is no dominant block in the remaining blocks. So for Img64cond, we first select Block0, but Block7 with CRM 10.46 is still dominant in remaining, so Block7 is included.

4.3 Quantitative Evaluations with FID

We tune the hyperparameters cfg (classifier free guidance), lr (denoise learning rate), attntemp (attention temperature), sampling 50000 images with "For" iteration to restore the FIDs results in [Zhai et al., 2024]. Treat it as the target FIDs, then keep the hyperparameters consistent, sampling with different GS-Jacobi strategies, recording the FIDs, relative error (%), running time (100 s) and accelerating rate. Rates with a relative error less than 1% are bolded. The strategy is as stated above [Stack-GS-J-Else]. All samplings are performed on 8 A800 GPUs with 80G memory.

In Table 3a Img128, keep "For" iteration for tough Block6, just few pure Jacobi for other blocks are enough to get good FID, like [6-1024-1-10], speeds up 3.67x. Then we fixed the total Jacobi times for Block6 with 128 and 256, and try different [GS-J] pairs. We found that simple strategies, like [6-8-32-10] can achieve results with relative error < 1% and surprising speed-up.

Similar results occurred in the sampling for AFHQ, as shown in Table 3c. Since the two models both have just one tough block, the acceleration rate behave similarly. For Img64 models, the situations are quite different. As shown in Table 3b Img64uncond, acceleration rates are not as high as single tough block models because it stacks both Block0 and 6, but still speeds up about 3x.

In Img64uncond, we treat two tough blocks equal since the CRMs have no absolute gap. For Img64cond, we first stack both Block0 and Block7 to original "For" loop, get the rate 2.31. Then we keep Block0 unchanged, try different strategies for Block7, the rate can be improved to 2.42. From

Table 3: FIDS of different GS-Jacobi strategies for four Models, with relative error <1% bolded.

Strategy	FID (rel)	time (rate)	Strategy	FID (rel)	time (rate)
Original	5.06	133.19 (1.00)	Original	14.67	109.05 (1.00)
Jacobi-30	10.36	58.16 (2.29)	Jacobi-30	25.66	45.60 (2.39)
Jacobi-60	6.07	114.24 (1.17)	Jacobi-60	17.98	92.06 (1.18)
[6-1024-1-8]	5.07 (0.20)	33.11 (4.02)	[0/6-1024-1-10]	15.27 (4.10)	38.17 (2.86)
[6-1024-1-10]	5.04 (0.00)	36.31 (3.67)	[0/6-1024-1-20]	14.77 (0.68)	47.65 (2.29)
[6-1024-1-20]	5.04 (0.00)	52.98 (2.51)	[0/6-1024-1-30]	14.72 (0.34)	57.16 (1.91)
[6-1-128-10]	5.50 (8.70)	48.46 (2.75)	[0/6-2-64-20]	15.22 (3.7)	51.11 (2.13)
[6-2-64-10]	5.19 (2.60)	34.08 (3.91)	[0/6-4-32-20]	15.18 (3.5)	36.89 (2.96)
[6-4-32-10]	5.22 (3.20)	27.59 (4.83)	[0/6-8-16-20]	16.44 (12.1)	28.11 (3.88)
[6-8-16-10]	5.40 (6.72)	24.37 (5.46)	[0/6-16-8-20]	21.16 (44.2)	26.14 (4.17)
[6-1-256-10]	5.30 (4.74)	78.93 (1.69)	[0/6-2-128-20]	15.03 (2.50)	76.41 (1.43)
[6-2-128-10]	5.10 (0.79)	48.75 (2.73)	[0/6-4-64-20]	14.81 (0.95)	50.06 (2.18)
[6-4-64-10]	5.05 (0.00)	35.64 (3.74)	[0/6-8-32-20]	14.80 (0.89)	36.84 (2.96)
[6-8-32-10]	5.09 (0.59)	29.41 (4.53)	[0/6-16-16-20]	15.17 (3.40)	31.38 (3.47)
[6-16-16-10]	5.16 (2.00)	26.38 (5.05)	[0/6-32-8-20]	17.47 (19.0)	29.86 (3.65)
(a) Img128cond with cfg=1.5 lr=0.97			(b) Img64uncond with cfg=0.2 attn=0.3 lr=0.9		
Strategy	FID (rel)	time (rate)	Strategy	FID (rel)	time (rate)
original	13.60	109.24 (1.00)	Original	4.42	12.16 (1.00)
[7-1024-1-10]	13.61 (0.07)	26.58 (4.11)	[0/7-256-1-6]	4.42 (0.00)	5.26 (2.31)
[7-1024-1-20]	13.60 (0.00)	37.69 (2.90)	[0/7-256-1-8]	4.42 (0.00)	5.81 (2.09)
[7-1024-1-30]	13.60 (0.00)	48.70 (2.24)	[0/7-256-1-10]	4.42 (0.00)	6.37 (1.91)
[7-1-128-10]	14.70 (8.08)	33.36 (3.27)	[0/7-256/1-1/64-6]	4.41 (0.00)	6.78 (1.79)
[7-2-64-10]	14.27 (4.92)	23.56 (4.54)	[0/7-256/2-1/32-6]	4.40 (0.00)	5.51 (2.20)
[7-4-32-10]	14.15 (4.04)	19.06 (5.73)	[0/7-256/4-1/16-6]	4.54 (2.71)	4.86 (2.50)
[7-8-16-10]	15.52 (14.1)	16.83 (6.49)	[0/7-256/8-1/8-6]	5.35 (21.0)	4.59 (2.65)
[7-1-256-10]	14.21 (4.48)	53.62 (2.04)	[0/7-256/4-1/24-6]	4.38 (0.00)	5.34 (2.28)
[7-2-128-10]	14.07 (3.45)	33.98 (3.21)	[0/7-256/8-1/13-6]	4.41 (0.00)	5.03 (2.42)
[7-4-64-10]	13.82 (1.62)	24.81 (4.40)	[0/7-16/8-8/13-6]	4.50 (1.81)	4.63 (2.63)
[7-8-32-10]	13.73 (0.96)	20.54 (5.32)	[0/7-16/8-10/13-6]	4.43 (0.23)	4.85 (2.51)
[7-16-16-10]	14.12 (3.82)	18.49 (5.91)	[0/7-16/8-12/13-6]	4.42 (0.00)	4.97 (2.45)
(c) AFHQ with cfg=3.4 lr=1.4			(d) Img64cond with cfg=2.0 lr=1.0		

Figure 5 and Table 2, we notice that Block0 behaves much tougher than any other blocks, so we segment Block0 into more modules and get $2.51\times$ speed up.

Based on all above experiments, we can conclude that, the fewer the blocks with dominant CRMs and the longer the time step after patching, the more significant the acceleration can achieve by GS-Jacobi sampling. This is consistent with intuition. GS-Jacobi achieves acceleration by iterating batches of equations in parallel, avoiding repeated serial updates of the kv caches in the "For" loop.

5 Conclusion

In this paper, we comprehensively optimize the sampling process of TarFlow models. By identifying the non-uniform transformation patterns across TarFlow blocks and proposing IGM and CRM, we effectively address the problems of initial value chosen and convergence rate differences. The introduction of the GS-Jacobi iteration and its in-depth error propagation analysis provides practical and efficient solution for TarFlow sampling. The experimental results on multiple TarFlow models show the superiority of proposed methods. The GS-Jacobi sampling achieving speed-ups of $4.53\times$ in Img128cond, $5.32\times$ in AFHQ, $2.96\times$ in Img64uncond, and $2.51\times$ in Img64cond without degrading sample quality, which is of great significance for the application of TarFlow models.

There are still some aspects that can be further improved. The strong assumption in calculating CRM needs more theoretical verification. In addition, the current method of determining GS - Jacobi parameters is relatively simple, and more intelligent and adaptive strategies are expected to be developed in the future.

References

- Hartwig Anzt, Edmond Chow, and Jack Dongarra. Iterative sparse triangular solves for preconditioning. In *Euro-Par 2015: Parallel Processing: 21st International Conference on Parallel and Distributed Computing, Vienna, Austria, August 24-28, 2015, Proceedings 21*, pages 650–661. Springer, 2015.
- Hartwig Anzt, Edmond Chow, Daniel B Szyld, and Jack Dongarra. Domain overlap for iterative sparse triangular solves on gpus. In *Software for Exascale Computing-SPPEXA 2013-2015*, pages 527–545. Springer, 2016.
- Jens Behrmann, Will Grathwohl, Ricky TQ Chen, David Duvenaud, and Jörn-Henrik Jacobsen. Invertible residual networks. In *International conference on machine learning*, pages 573–582. PMLR, 2019.
- Siavash A Bigdeli, Geng Lin, L Andrea Dunbar, Tiziano Portenier, and Matthias Zwicker. Learning generative models using denoising density estimators. *IEEE Transactions on Neural Networks and Learning Systems*, 2023.
- Joey Bose, Ariella Smofsky, Renjie Liao, Prakash Panangaden, and Will Hamilton. Latent variable modelling with hyperbolic normalizing flows. In *International conference on machine learning*, pages 1045–1055. PMLR, 2020.
- Yunjey Choi, Youngjung Uh, Jaejun Yoo, and Jung-Woo Ha. Stargan v2: Diverse image synthesis for multiple domains, 2020. URL <https://arxiv.org/abs/1912.01865>.
- Edmond Chow, Hartwig Anzt, Jennifer Scott, and Jack Dongarra. Using jacobi iterations and blocking for solving sparse triangular systems in incomplete factorization preconditioning. *Journal of Parallel and Distributed Computing*, 119:219–230, 2018.
- Nicola De Cao, Wilker Aziz, and Ivan Titov. Block neural autoregressive flow. In *Uncertainty in artificial intelligence*, pages 1263–1273. PMLR, 2020.
- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.
- Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.
- Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real nvp. *arXiv preprint arXiv:1605.08803*, 2016.
- Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. Made: Masked autoencoder for distribution estimation. In *International conference on machine learning*, pages 881–889. PMLR, 2015.
- Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.
- Matej Grčić, Ivan Grubišić, and Siniša Šegvić. Densely connected normalizing flows. *Advances in Neural Information Processing Systems*, 34:23968–23982, 2021.
- Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

- Jonathan Ho, Xi Chen, Aravind Srinivas, Yan Duan, and Pieter Abbeel. Flow++: Improving flow-based generative models with variational dequantization and architecture design. In *International conference on machine learning*, pages 2722–2730. PMLR, 2019.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Chin-Wei Huang, David Krueger, Alexandre Lacoste, and Aaron Courville. Neural autoregressive flows. In *International conference on machine learning*, pages 2078–2087. PMLR, 2018.
- Carl T Kelley. *Iterative methods for linear and nonlinear equations*. SIAM, 1995.
- Durk P Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. *Advances in neural information processing systems*, 31, 2018.
- Durk P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. *Advances in neural information processing systems*, 29, 2016.
- Maxim Naumov. Parallel complexity of forward and backward propagation. *arXiv preprint arXiv:1712.06577*, 2017.
- Aaron Oord, Yazhe Li, Igor Babuschkin, Karen Simonyan, Oriol Vinyals, Koray Kavukcuoglu, George Driessche, Edward Lockhart, Luis Cobo, Florian Stimberg, et al. Parallel wavenet: Fast high-fidelity speech synthesis. In *International conference on machine learning*, pages 3918–3926. PMLR, 2018.
- James M Ortega and Werner C Rheinboldt. *Iterative solution of nonlinear equations in several variables*. SIAM, 2000.
- George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation. *Advances in neural information processing systems*, 30, 2017.
- Massimiliano Patacchiola, Aliaksandra Shysheya, Katja Hofmann, and Richard E Turner. Transformer neural autoregressive flows. *arXiv preprint arXiv:2401.01855*, 2024.
- Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. Sdxl: Improving latent diffusion models for high-resolution image synthesis. *arXiv preprint arXiv:2307.01952*, 2023.
- Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.
- Andrea Santilli, Silvio Severino, Emilian Postolache, Valentino Maiorca, Michele Mancusi, Riccardo Marin, and Emanuele Rodolà. Accelerating transformer inference for translation via parallel decoding. *arXiv preprint arXiv:2305.10427*, 2023.
- Yang Song, Chenlin Meng, and Stefano Ermon. Mintnet: Building invertible neural networks with masked convolutions. *Advances in Neural Information Processing Systems*, 32, 2019.
- Yang Song, Chenlin Meng, Renjie Liao, and Stefano Ermon. Accelerating feedforward computation via parallel nonlinear equation solving. In *International Conference on Machine Learning*, pages 9791–9800. PMLR, 2021.
- Brian L Trippe and Richard E Turner. Conditional density estimation with bayesian normalising flows. *arXiv preprint arXiv:1802.04908*, 2018.
- Aäron Van Den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *International conference on machine learning*, pages 1747–1756. PMLR, 2016.
- Jay Whang, Erik Lindgren, and Alex Dimakis. Composing normalizing flows for inverse problems. In *International Conference on Machine Learning*, pages 11158–11169. PMLR, 2021.
- Shuangfei Zhai, Ruixiang Zhang, Preetum Nakkiran, David Berthelot, Jiatao Gu, Huangjie Zheng, Tianrong Chen, Miguel Angel Bautista, Navdeep Jaitly, and Josh Susskind. Normalizing flows are capable generative models. *arXiv preprint arXiv:2412.06329*, 2024.

A Convergence and Error Propagation

Indeed, (4) is an equivalent form of the diagonal Newton method. Let $f(X) = X - g(X)$, to find its root, the iteration of diagonal Newton method is:

$$X^{(k+1)} = X^{(k)} - D_f^{-1}(X^{(k)})f(X^{(k)})$$

with $D_f^{-1}(X^{(k)})$ the diagonal of Jacobi matrix $J_f(X) = \partial f / \partial X$. Because $g(X)$ is causal designed in T dimension, $D_f^{-1}(X^{(k)}) = I$. Then the iteration formula of diagonal Newton method is again (4).

Since $I - D_f^{-1}J_f$ is a strictly lower triangle matrix with zero spectral norm, the fixed point iteration (diagonal Newton method) is superlinear convergence in the neighbor of X^* . Indeed, this iteration can converge strictly after $T - 1$ times:

$$x_t^{(k+1)} - x_t^{(k)} = (\sigma_t^{(k)} - \sigma_t^{(k-1)})z_t + (u_t^{(k)} - u_t^{(k-1)})$$

Initially, $x_1^* = z_1$, for $t = 2$, one iteration can get the accurate x_2^* , and so for x_t^* . But we don't need a absolutely accurate solution, since the difference between $x_t^{(k)}$ and x_t^* reduced after each iteration. Let $\varepsilon^{(k)} = X^{(k)} - X^*$ be the error after k iteration, $f_t^{(k)} = x_t^{(k)} - \sigma_t^{(k)}z_t - u_t^{(k)}$, for t -th component:

$$\begin{aligned} e_t^{(k+1)} &= e_t^{(k)} - f_t^{(k)} \\ &\approx e_t^{(k)} - \sum_{i=1}^t \frac{\partial f_t}{\partial x_i} \Big|_{X^{(k)}} e_i^{(k)} \\ &= - \sum_{i=1}^{t-1} \frac{\partial f_t}{\partial x_i} \Big|_{X^{(k)}} e_i^{(k)} \end{aligned}$$

with the $\approx$ obtained by first-order Taylor expansion at X^* . Denote $\frac{\partial f_t}{\partial x_i} \Big|_{X^{(k)}}$ as $\gamma_{ti}^{(k)}$, then $e_t^{(k+1)} \approx - \sum_{i=1}^{t-1} \gamma_{ti}^{(k)} e_i^{(k)}$, and the error recursion can be written in a matrix form:

$$\varepsilon^{(k+1)} = \Gamma^{(k)} \varepsilon^{(k)}, \Gamma^{(k)} = - \begin{bmatrix} 0 & & & \\ \gamma_{21} & 0 & & \\ \vdots & \ddots & \ddots & \\ \gamma_{T1} & \dots & \gamma_{T,T-1} & 0 \end{bmatrix}^{(k)}. \quad (7)$$

Since $\varepsilon^{(k)} = \prod_{j=0}^{k-1} \Gamma^{(j)} \varepsilon^{(0)}$ and $e_1^{(0)} = 0$, the product of $\Gamma^{(k)}$ will move down the lower triangle part one unit after each iteration, so the error will go to 0 after $T - 1$ iteration.

Thus for each component e_t :

$$e_t^{(k)} = - \sum_{i=k+1}^{t-1} \gamma_{ti}^{(k)} e_i^{(k)} \quad (8)$$

B Derivation for Convergence Ranking Metric

To measure the convergence difference between TarFlow blocks, a direct method is to calculate the norm of error recursion matrix (7). But, analytically,

$$\begin{aligned}\gamma_{ti} &= -\frac{\partial \sigma_t}{\partial x_i} z_t - \frac{\partial u_t}{\partial x_i} \\ &= -\sigma_t z_t \frac{\partial s(x_{<t})}{\partial x_i} - \frac{\partial u(x_{<t})}{\partial x_i}\end{aligned}$$

the derivative of σ_t, u_t is hard to calculate since they are generated by series of attention layers, thus we propose a simple but valid alternatives. Since $s(x_{<t}), u(x_{<t})$ consist of series of attention layers and a project out layer:

$$\begin{aligned}s(x_{<t}) &= W_s \text{attn}(x_{<t}) + b_s \\ u(x_{<t}) &= W_u \text{attn}(x_{<t}) + b_u\end{aligned}$$

then:

$$\begin{aligned}\gamma_{ti} &= -(\sigma_t z_t W_s + W_u) \frac{\partial \text{attn}(x_{<t})}{\partial x_i} \\ \Gamma &= -(\Sigma(X) Z W_s + W_u) J_{\text{Attn}}(X)\end{aligned}$$

Now we make a very strong assumption: the norm of Jacobi matrix of attention layers between TarFlow blocks behave similar, or in coordination with the previous item, which can be dropped out. It's hard to strictly verify this but it holds in the experiments. Intuitively, this may be because the attention layers add various regularizations thus make the values bounded, and secondly, the changes in attention (i.e. derivatives) of TarFlow blocks with large changes are also large.

In practice, we can replace the non-volume-projection parts with $\Sigma^{-1}(X)X$ to calculate it at the same time with IGM in the forward direction. Intuitively $\Sigma(X)Z$ and $\Sigma^{-1}(X)X$ measure the same property from symmetric direction. Then the simplified norm of error can be written as:

$$\text{CRM} = \|\Sigma^{-1}(X)X\|_2 \|W_s\|_2 + \|W_u\|_2$$

C Components of Convergence Ranking Metric

The components of CRM for Non-Volume-Preserving $||\Sigma^{-1}X||$, variance $||W_s||$ and mean $||W_u||$ are shown as Table 4.

Table 4: Components of Convergence Ranking Norm, with dominant CRMs bolded.

	Imagenet128				AFHQ			
	$ \Sigma^{-1}X $	$ W_s $	$ W_u $	CRM	$ \Sigma^{-1}X $	$ W_s $	$ W_u $	CRM
Block0	20.06	0.31	0.24	6.52	52.78	0.96	0.86	51.85
Block1	14.77	0.46	0.18	7.03	39.32	1.28	0.75	51.45
Block2	13.65	0.19	0.37	3.08	34.70	1.90	0.73	66.76
Block3	33.65	0.40	0.17	13.63	73.39	0.87	0.65	64.98
Block4	43.65	0.21	0.26	9.66	118.90	0.61	0.94	73.77
Block5	53.86	0.16	0.26	9.17	159.91	0.51	0.96	84.05
Block6	131.84	0.53	0.19	70.54	153.54	0.49	0.94	76.64
Block7	26.08	0.18	0.17	5.05	286.83	1.21	0.93	348.51

	Img64uncond				Img64cond			
	$ \Sigma^{-1}X $	$ W_s $	$ W_u $	CRM	$ \Sigma^{-1}X $	$ W_s $	$ W_u $	CRM
Block0	53.48	0.41	0.10	22.29	82.72	1.70	0.42	141.22
Block1	8.00	0.11	0.12	1.06	14.26	0.59	0.78	9.25
Block2	8.55	0.08	0.24	1.01	4.53	0.16	0.59	1.36
Block3	13.79	0.09	0.12	1.48	7.61	0.16	0.53	1.82
Block4	8.36	0.06	0.25	0.77	25.09	0.29	0.33	7.68
Block5	8.06	0.05	0.13	0.58	27.44	0.17	0.38	5.08
Block6	40.50	0.36	0.13	14.78	12.26	0.21	0.45	3.08
Block7	13.97	0.12	.26	1.95	56.91	0.34	0.28	19.81

D IGM and CRM with Different Norm

We also computed the IGM and CRM under both the Frobenius norm and the 1-norm, shown in Table 5a 5b. The numerical values differ among the Frobenius norm (F), 1-norm, and spectral norm (Table 1 2), while the relative ranks of each block remain entirely consistent. Notably, the spectral norm exhibits a more dispersed distribution in its measurements.

Table 5: IGM and CRM for four models in both F-Norm and 1-Norm

Models	Img128cond		Img64cond		Img64uncond		AFHQ	
F-norm	Z	Z_0	Z	Z_0	Z	Z_0	Z	Z_0
Block0	12.40	14.61	4.2×10^5	10.13	1416.83	8.29	21.38	23.24
Block1	14.83	3.92	7.95	8.66	10.67	7.08	12.79	11.23
Block2	3.56	6.50	6.55	7.66	9.39	7.83	13.12	11.69
Block3	10.27	13.94	5.00	7.42	6.47	5.30	35.07	44.06
Block4	9.37	29.58	10.63	13.86	4.02	10.19	16.39	53.62
Block5	14.88	27.92	5.39	26.29	3.86	7.82	30.00	54.26
Block6	53.36	41.41	4.54	19.83	33.96	7.74	14.99	60.40
Block7	10.27	39.87	24.04	14.17	24.98	35.99	134.16	145.32
1-Norm	Z	Z_0	Z	Z_0	Z	Z_0	Z	Z_0
Block0	137.76	198.3	3.5×10^6	39.77	14915.71	156.51	94.23	148.39
Block1	167.96	53.21	39.2	49.39	204.01	116.13	71.84	113.24
Block2	88.66	99.38	35.58	45.56	143.53	98.95	204.35	69.05
Block3	106.1	161.9	49.86	42.7	188.01	99.36	302.48	277.35
Block4	97.83	551.18	95.01	156.55	61.84	169.69	158.54	1024.6
Block5	151.26	357.02	48.15	173.57	68.53	112.22	271.19	429.51
Block6	634.4	454.57	38.66	228.86	1028.43	174.63	99.23	766.3
Block7	118.41	449.14	236.01	77.94	524.11	633.18	1698.14	1370.99

(a) IGM with Frobenius Norm and 1-Norm

Models	Img128cond		Img64cond		Img64uncond		AFHQ	
F-Norm	CRM	Percent	CRM	Percent	CRM	Percent	CRM	Percent
Block0	9.35	5.50	150.29	63.42	19.23	35.22	60.31	5.10
Block1	10.89	6.41	14.62	6.17	2.12	3.88	74.53	6.30
Block2	5.72	3.37	3.66	1.54	2.25	4.13	103.02	8.72
Block3	17.12	10.08	4.59	1.93	3.12	5.72	82.16	6.95
Block4	16.50	9.71	13.09	5.52	1.68	3.08	104.05	8.80
Block5	19.34	11.38	10.82	4.57	1.43	2.62	133.83	11.33
Block6	83.75	49.30	6.59	2.78	21.99	40.27	149.97	12.69
Block7	7.17	4.22	33.25	14.03	2.75	5.04	473.30	40.07
1-Norm	CRM	Percent	CRM	Percent	CRM	Percent	CRM	Percent
Block0	99.98	3.39	799.34	45.24	219.04	25.17	1320.45	4.56
Block1	194.08	6.59	274.96	15.56	46.72	5.37	948.31	3.28
Block2	84.62	2.87	15.43	0.87	25.37	2.91	1085.57	3.75
Block3	666.29	22.64	33.05	1.87	64.74	7.44	2364.8	8.18
Block4	160.53	5.45	97.42	5.51	15.79	1.81	1799.76	6.22
Block5	345.13	11.72	144.68	8.18	23.56	2.70	4486.86	15.52
Block6	1253.95	42.61	52.84	2.99	341.51	39.25	2396.26	8.28
Block7	137.96	4.68	349.06	19.75	133.23	15.31	14507.4	50.18

(b) CRM with Frobenius Norm and 1-Norm

E GS-Jacobi Sampling

The complete algorithm of GS-Jacobi sampling is as follows:

Algorithm 1 Gauss-Seidel-Jacobi Sampling

Input: Well trained TarFlow model containing L blocks $\{\text{Block}^{(l)} := \Sigma^{(l)}, \mu^{(l)}\}_{l=1}^L$, a batch of training samples X , a batch of noise Z , other hyperparameters.

Output: Generated images of the same size as Z .

Preprocessing:

- 1: Patchify X into size (B, T, C)
- 2: **for** $\text{Block}^{(l)}$, l in $1 : L$ **do**
- 3: Calculate $\text{IGM}^{(l)}$ with equation (5)
- 4: Calculate $\text{CRM}^{(l)}$ with equation (6)
- 5: **end for**
- 6: Record the initial guessing mode of l -th block to $Z^{(l)}$ or $Z_0^{(l)}$ according to $\text{IGM}^{(l)}$
- 7: Determine the GS modules numbers $\{G_l\}_{l=1}^L$ and Jacobi times $\{J_l\}_{l=1}^L$ for each blocks according to $\text{CRM}^{(l)}$, with $J_L \leq T/G_L$

Sampling:

- 1: Patchify Z into size (B, T, C)
- 2: Set $Z^{(1)} = Z$, $\text{ebound} = 10^{-8}$
- 3: **for** $\text{Block}^{(l)}$, l in $1 : L$ **do**
- 4: **for** g in $1 : G_l$ **do**
- 5: set $k = 0, e = 1000$
- 6: **while** $k < J_l$ and $e > \text{ebound}$ **do**
- 7: $X_g^{(l)(k+1)} = \Sigma_g(X_{:g}^{(l)(k)})Z_g^{(l)} + \mu_g(X_{:g}^{(l)(k)})$
- 8: $e = \|X_g^{(l)(k+1)} - X_g^{(l)(k)}\| / (B \times T \times C)$
- 9: $k = k + 1$
- 10: **end while**
- 11: $Z^{(l+1)} = X^{(l)(k+1)}$
- 12: **end for**
- 13: **end for**
- 14: **return** Unpatchified $Z^{(L+1)}$

Jacobi计算

根据 CRM决定 最大迭代次数(J)和 J块个数(GS),
GS * J <= L

An intuition Figure is:

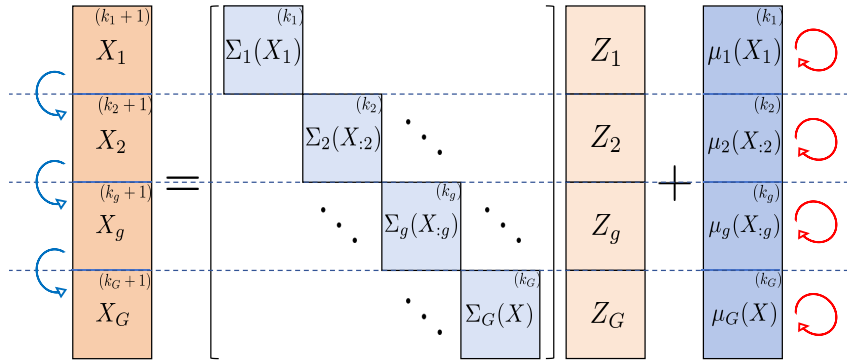


Figure 6: Intuition diagram of Gauss-Seidel-Jacobi sampling in single block. The horizontal long dashed line segment (4) into G subgroups. The red rotating arrow denote the in-group Jacobi iteration with $k_g \leq \text{card}(\mathcal{G}_g)$ times. Then, the solution closed to X_g^* will be delivered to next subgroup serially, this is the Gauss-Seidel part which denoted by blue rotating arrow.

F Visual Comparison of Different Methods

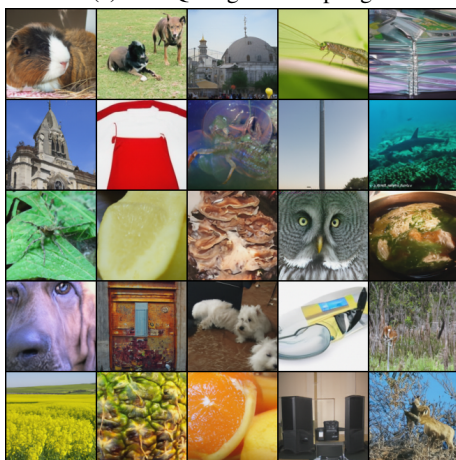
We used the original "For" loop and GS-Jacobi strategy with FID relative error within 1% for each model under the same guidance and denoise, and visualized the sampling results in Figure 7.



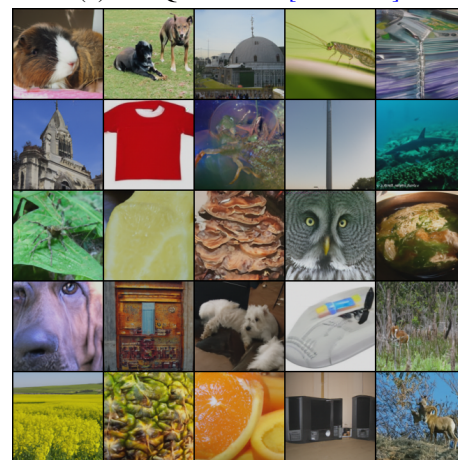
(a) AFHQ Original Sampling



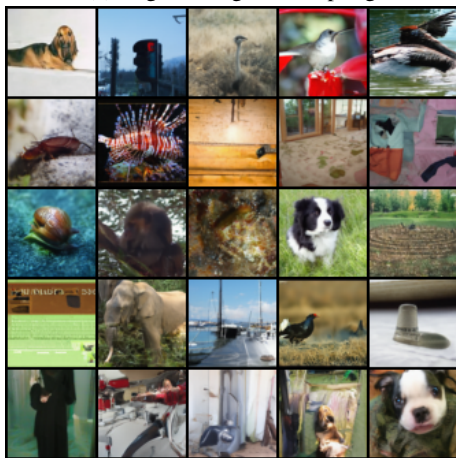
(b) AFHQ GS-Jacobi [7-8-32-10]



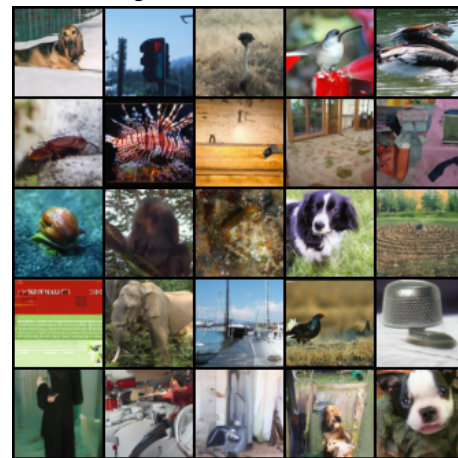
(c) Img128 Original Sampling



(d) Img128 GS-Jacobi [6-8-32-10]



(e) Img64 Original Sampling



(f) Img64 GS-Jacobi [0/7-16/8-10/13-6]

Figure 7: Visual Comparison of Different Methods